Predicting Income from Census Data: A Supervised Learning Proof-of-Concept

Group members: [Names]

October 17, 2026

Dataset used: Adult / Census Income (OpenML, `name="adult"`, version 2).

1 Problem Framing and Exploratory Data Analysis

1.1 Task and train/test split

The task is to predict whether an individual earns more than $50,000 per year from 14 demographic and employment attributes of the 1994 US Census [1]. The positive class (>50K) represents 23.9% of the observations, so the problem is imbalanced (Section 3).

The data are split once into a training set (80%, 39,032 rows) and a test set (20%, 9,758 rows), with stratification on the target and a fixed seed; both sets contain 23.9% of positives. The test set is not used before the final evaluation: model comparisons rely on 5-fold stratified cross-validation within the training set, and every relationship between a feature and the target reported below is computed on the training set only.

1.2 Data quality

Duplicates and removed columns. The raw data contain 48,842 rows, of which 52 are exact duplicates; these are removed, leaving 48,790 observations. This is done before dropping any column: if `fnlwgt` were removed first, 6,374 rows would appear duplicated, although they are distinct individuals who share the same profile, not recording errors. Two columns are then removed. `fnlwgt` is a sampling weight indicating how many people in the US population a respondent represents; it is not linked to the individual's characteristics. `education_num` maps one-to-one onto `education` and is therefore redundant.

Missing values. Three columns contain missing values: `workclass` (5.7%), `occupation` (5.7%) and `native_country` (1.8%). 10 rows with no occupation belong to the `Never-worked` class, so they have no occupation by definition. Then we have 2,795 rows where both `workclass` and `occupation` are missing together.

To choose how to handle them, we need to know whether these values are missing at random [2]. We cannot know why a value is missing, but we can compare people with and without a reported work class (Table 1).

People with a missing work class are much more often very young or old, work part time more often, and more often live with their parents. This profile suggests people outside regular employment, such as students and retirees. As a result, they earn >50K much less often (9.4% against 24.8%). Even among people aged 21 to 64, the gap remains (10.9% against 26.7%), so age alone does not explain it. The missing values of `workclass` and `occupation` are therefore not random: their absence is probably linked to the person's situation and to income.

Table 1: Profile of people with a missing or reported work class (training set).

	Work class missing	Work class reported
Aged 20 or younger, or 65 or older	39.2%	9.9%
Works less than 35 hours per week	44.7%	15.6%
Lives with parents (`Own-child`)	30.4%	14.7%
Earns >50K	9.4%	24.8%
Earns >50K, aged 21 to 64 only	10.9%	26.7%

The case of `native_country` is different. People with a missing country earn >50K in 25.1% of cases, close to the overall rate of 23.9%. Their absence does not seem linked to the person's situation.

Extreme values. We examined the four numerical variables in two ways: a standard outlier rule and the maximum values.

First, we applied Tukey's rule that flags values below $Q_1 - 1.5\,\text{IQR}$ or above $Q_3 + 1.5\,\text{IQR}$. For age, the rule flags 0.4% of people, all older than 78. These are real elderly people, not errors. For weekly hours, it flags 27.6% of people. This happens because 46.7% of people work exactly 40 hours, so $Q_1 = 40$, $Q_3 = 45$, and the bounds are 32.5 and 52.5 hours. As Figure 1b shows, the rule then flags wrongly normal part-time jobs and long working weeks. Regarding capital income, most values are zero (91.7% for gains and 95.3% for losses). Therefore, $Q_1 = Q_3 = 0$ and every non-zero value would be flagged. The non-zero values are also very skewed (handled in section 2.2): the median gain is 7,298, but the maximum is 99,999 (Appendix A).

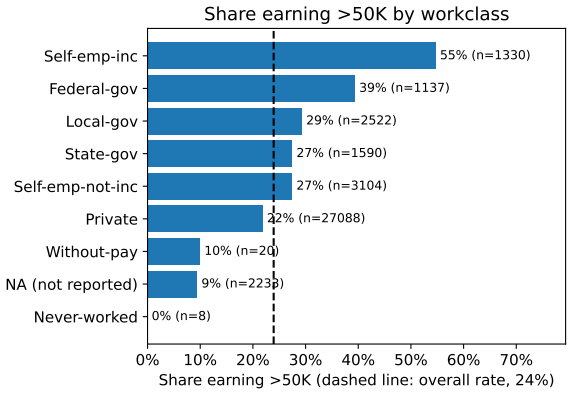
Second, the maximum values show that some variables are capped. 54 people are exactly 90 years old, against 6 and 2 for ages 88 and 89. 137 people work exactly 99 hours per week. 244 people have a capital gain of exactly 99,999, while the next highest gain is 41,310. These values probably mean "this amount or more" and are not errors. For instance, the 99,999 value is very informative: 100% of these people earn >50K.

Rare and inconsistent categories. Some categories contain very few people. In the training set, `Never-worked` has 8 people, `Without-pay` 20 and `Armed-Forces` 12. In the full dataset, 17 countries have fewer than 50 people. Rates computed on such small groups are unreliable. We also found four inconsistent rows: one woman is recorded as a `Husband` and three men as a `Wife`.

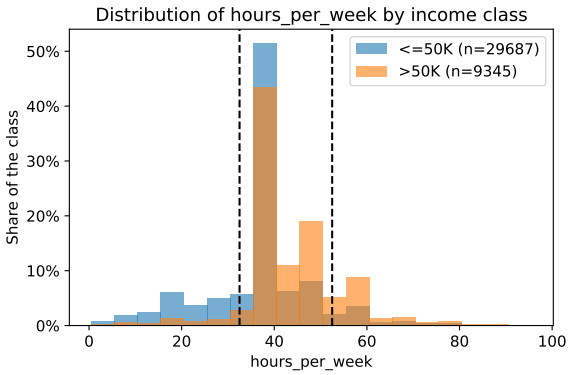
1.3 Relationships with the target

Employment. Income varies strongly with the type of employment (Figure 1a). The private sector covers 69% of people, and 22% of them earn >50K. The rate is higher for federal employees (39%) and highest for self-employed people with their own company (55%). People with a missing work class have a low rate (9%), in line with the profile of Table 1. Working time also matters (Figure 1b): 40.2% of people who work more than 40 hours earn >50K, against 9.4% of people who work less than 40 hours.

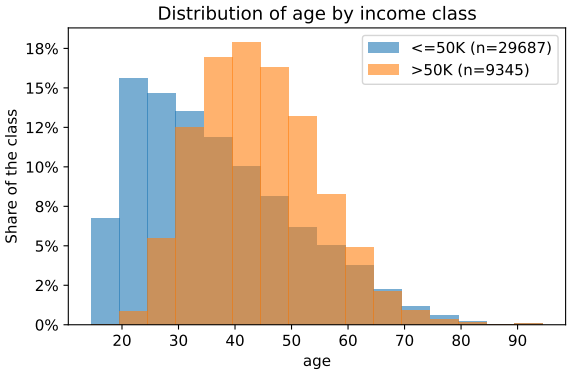
Demographics. High earners are older: their median age is 43, against 34 for the others (Figure 1c), and almost no one under 25 earns >50K (1.2%). Marital status also matters. Married people living with their spouse earn >50K in 44.7% of cases, while the other frequent statuses stay between 4.5% and 10.1% (Figure 1d). Education shows a similar gap, from 1.6% (`Preschool`) to 74.1% (`Doctorate`). Finally, 21.3% of high earners have a capital gain, against 4.1% of the others.



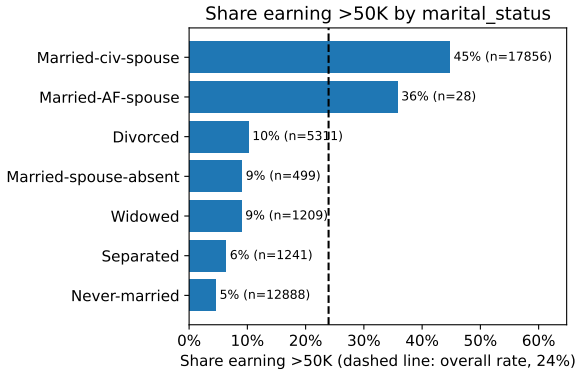
(a) Share of >50K by work class



(b) Weekly hours by class



(c) Age by class



(d) Share of >50K by marital status

Figure 1: Exploratory analysis on the training set. (a) and (d): share of people earning >50K in each category; the dashed line is the overall rate. In (a), missing work classes are shown as a separate bar. (b) and (c): distribution within each class (each class sums to 100%). In (b), the dashed lines are Tukey's bounds: $40 - 1.5 \times 5 = 32.5$ and $45 + 1.5 \times 5 = 52.5$ hours.

Sex. The share of high earners differs by sex: 30.3% for men against 11.1% for women. In addition, `relationship` is closely linked to sex: a `Husband` is a man in all cases but one, and a `Wife` is a woman in all cases but three. This link will be important for our fairness audit.

2 Data Preparation and Feature Engineering

2.1 Missing values

Section 1.2 showed that the missing values of `workclass` and `occupation` are not random. We considered three options. Deleting these rows would remove 5.7% of the data and, above all, a specific group of low earners, which would bias the model. Replacing them with the most frequent value (`Private`) would mix two groups with different incomes (9.4% against 21.9%). We therefore keep them as a separate category, called `Missing`, in both columns. The model can then learn what the absence means.

For `native_country`, the share of high earners (25,1%) is similar to the overall share (23,9%), which suggests that this missingness is unrelated to the target, and it concerns only 1.8% of rows. We therefore replace these values with the most frequent country, the United States, which covers 91.4% of people whose country is known. It is the most likely value, so the risk of error is small.

Both treatments are applied inside the pipeline: the most frequent value is computed on the

training folds only, so no information comes from the data used for evaluation.

2.2 Extreme values

We do not remove any extreme value, for two reasons: none of them is an error and removing them would lose information, especially the capital gain of 99,999, which always means >50K.

The remaining question is whether these values can harm the models. Tree-based models are not affected, because they split the data with thresholds. The logistic regression is more sensitive so for this model, numerical features are standardised and capital income is log-transformed (Section 2.5), which reduces the weight of very large values.

2.3 Rare and inconsistent categories

Regarding the rare categories, we keep them. Most of them disappear in the engineered features of Section 2.5: rare countries are grouped in `native_us`, and `Married-AF-spouse` is merged into `is_married`. The others, such as `Never-worked`, are too small to influence the models. Regarding the four inconsistent rows, we also keep them. They represent less than 0.01% of the data and cannot affect the results, while correcting them would require guessing which value is wrong.

2.4 Encoding and scaling

Models need numerical inputs. The six categorical variables have no natural order, so we use one-hot encoding: each category becomes a 0/1 column. This concerns `workclass`, `education`, `occupation`, `relationship`, `race` and `sex`. Very rare categories not seen during training are ignored in the encoder. We keep one column per category. Although this creates one redundant column per variable, it is not a problem: the logistic regression uses L2 regularisation by default, and trees are not affected. Numerical features are standardised. All models use the same pipeline, so that Section 4 compares the models only.

2.5 Engineered features

We create three features, each replacing raw columns (Table 2).

Table 2: Engineered features and the columns they replace.

Feature	Replaces	Definition	Justification
`capital_net`	`capital_gain`, `capital_loss`	$\text{sign}(g-\ell)\log(1+\lvert g-\ell\rvert)$, with g the gain and ℓ the loss	No one has both a gain and a loss, so one column keeps all the information. The log reduces the skew (Appendix A).
`is_married`	`marital_status`	1 if married and living with spouse	The income gap is concentrated in this group: 44.7% against at most 10.1% for the other frequent statuses.
`native_us`	`native_country`	1 if born in the United States	Most countries are too small for reliable rates, and the gaps between countries mostly follow education (see below).

Reducing `native_country` to a binary `native_us` indicator may appear to discard information. For instance, 5.1% of people born in Mexico earn >50K, against 42.9% of people born in India. However, we still keep a single indicator for two reasons. First, most countries are too small: 17 countries have fewer than 50 people, so their rates are unreliable. Second, the gaps

between countries mostly follow education. Across the 24 countries with at least 50 people, the share of high earners and the share of university graduates (`Bachelors` or higher) have a correlation of 0.87. Since education is already in the model, a detailed country variable would add little. The indicator still keeps the main difference: 24.4% of people born in the United States earn >50K, against 18.4% of the others. Note that this correlation is computed on country averages, so it shows an association, not a causal link.

3 Class Imbalance

3.1 Metrics

The positive class represents only 23.9% of the data. With such an imbalance, accuracy is misleading: a model that always predicts ≤50K reaches 76.1% accuracy without finding a single high earner. We therefore do not use accuracy, and choose metrics that focus on the positive class.

Our main metric is the area under the precision–recall curve (PR-AUC). It measures how well the model ranks high earners above the others, over all possible thresholds. It focuses on the positive class, and a random model scores 0.239, the share of positives [3]. We also report ROC-AUC, the standard ranking metric, but only as a secondary metric. ROC-AUC relies on the false positive rate, FP/(FP + TN). Since true negatives are numerous, this rate stays low even when the model makes many false alarms, so ROC-AUC can look better than the model really is. Our imbalance is moderate, so this effect is smaller than in extreme cases, but it remains visible: in Section 4, ROC-AUC is always much higher than PR-AUC.

Ranking metrics do not show the decisions the model actually makes. We therefore also report three metrics at the default threshold of 0.5: recall, the share of high earners that the model finds; precision, the share of predicted high earners that are correct; and F1, which balances the two.

3.2 Chosen remedy

We handle the imbalance with class weighting. This method changes the training loss: errors on high earners count more, so the model cannot ignore them. It does not change the data and works inside the pipeline without any risk of leakage.

In the code, the weight is passed to each model as `class_weight={0: 1, 1: w}`: an error on a high earner counts w times more than an error on a low earner. This is equivalent to the `scale_pos_weight` parameter seen in class. We use `class_weight` because it is available in all three of our models, so the imbalance is handled in the same way for each of them.

The best value of w is not known in advance. We therefore treat it as a hyperparameter and tune it with Optuna (Section 6), between 1 and $n_0/n_1 \approx 3.18$, where n_0 and n_1 are the numbers of negatives and positives in the training set. The lower bound, $w = 1$, means no weighting: it leaves Optuna free to conclude that weighting is not useful.

4 Modelling Progression

4.1 Evaluation protocol

The goal of this section is to choose the model family, not its final settings. All models therefore keep their default hyperparameters. Each model is placed in the same pipeline (Section 5) and evaluated with 5-fold stratified cross-validation on the training set. The training set is split into five parts that keep the class ratio. Each part is used once for evaluation, while the model is trained on the other four. We report the mean score over the five folds. To detect overfitting, we

also compute the PR-AUC on the training folds: a large gap with the cross-validated PR-AUC means that the model fits the training data too closely.

4.2 Models

We compare three models of increasing flexibility.

Baseline: logistic regression. It is a simple linear model, fast and easy to interpret. It sets the minimum level that more complex models must beat.

Random forest. It averages many decision trees, each trained on a random sample of the data. Trees can capture non-linear effects and interactions between features, and averaging them reduces the instability of a single tree.

Gradient boosting with LightGBM. It builds trees one after another, each new tree correcting the errors of the previous ones. Among boosting libraries, we chose LightGBM over XGBoost and CatBoost. Our dataset is of moderate size and we have no strong resource constraint, so raw performance does not clearly separate these libraries. Essentially, we chose LightGBM because it was covered in class and because it accepts the same `class_weight` parameter as the scikit-learn models, so that the imbalance is handled identically for all models.

4.3 Results

Table 3: Mean scores over 5-fold cross-validation on the training set (default hyperparameters, threshold 0.5). The last column is the PR-AUC on the training folds.

Model	Precision	Recall	F1	ROC-AUC	PR-AUC	Train PR-AUC
Logistic regression	0.700	0.566	0.626	0.890	0.709	0.711
Random forest	0.695	0.618	0.654	0.889	0.731	0.991
LightGBM	**0.781**	**0.658**	**0.714**	**0.929**	**0.830**	0.860

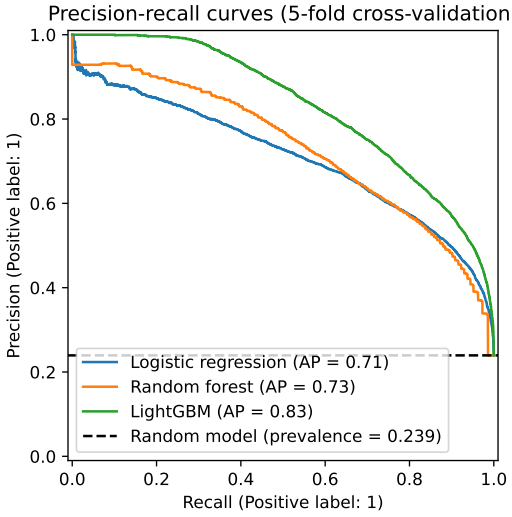


Figure 2: Precision–recall curves from out-of-fold predictions (5-fold cross-validation on the training set). AP is the average precision, i.e. the PR-AUC. The dashed line is the score of a random model.

Ranking of the models. LightGBM is clearly the best model (Table 3). Its PR-AUC (0.830) is 12.1 points higher than the baseline, and it also has the best ROC-AUC, precision, recall and F1. Figure 2 confirms this: its curve lies above the two others at almost every level of recall. The random forest improves on the baseline only slightly (+2.2 points of PR-AUC), and the two models have the same ROC-AUC (0.889 and 0.890). Their curves are close, and they meet at high recall. All models are far above the random model (0.239), so they all learn a real signal.

Overfitting. The gap between training and cross-validated PR-AUC differs strongly between models. The logistic regression shows almost no gap (0.711 against 0.709). It does not overfit, but its simple linear form limits its score. The random forest shows a very large gap (0.991 against 0.731). With default settings, its trees grow until each leaf is almost pure, so the model learns the training data almost by heart. This overfitting probably explains why it barely beats the baseline despite its flexibility. LightGBM shows a moderate gap (0.860 against 0.830). By default, its trees are limited to 31 leaves and each tree only corrects part of the error, which keeps its complexity under control. LightGBM is also the most stable model across folds: the standard deviation of its PR-AUC is 0.002, against 0.006 for the random forest and 0.009 for the logistic regression.

Decisions at the default threshold. For all three models, precision is higher than recall. At the threshold of 0.5, the models miss between 34% and 43% of high earners. This is expected without class weighting: low earners dominate the training loss, so the models predict $\leq$50K too often. This is precisely what the tuning of w will address (Section 3.2).

Conclusion. LightGBM is our main candidate. All three models are nevertheless tuned in Section 6, so that each model family is compared at its best. The random forest in particular should benefit from tuning: its gap between training and cross-validation suggests that limiting the growth of its trees could improve it.

5 Pipeline

All preparation steps and the model are wrapped in a single scikit-learn `Pipeline` (function `build_pipeline` in `src/pipeline.py`). The pipeline takes the raw data as input and returns a prediction. It chains the three steps of Table 4.

Table 4: Steps of the pipeline.

Step	What it does	Learned from the data
1. Feature engineering	Creates `capital_net`, `is_married` and `native_us`, and removes the columns they replace (Section 2.5)	Nothing
2. Preprocessing	Numerical features: imputation with the most frequent value, then standardisation. Categorical features: `Missing` category, then one-hot encoding	Most frequent values, means, standard deviations, list of categories
3. Model	Logistic regression, random forest or LightGBM	The model parameters

Using a single pipeline has three benefits. First, it prevents data leakage. During cross-validation, the whole pipeline is fitted again on the training folds of each split. The values learned in steps 2 and 3, such as means or most frequent values, therefore never use the evaluation fold. Step 1 learns nothing, because each new value only uses the person's own row, but placing it in the pipeline means that new raw data are processed automatically. Second, the three models

receive exactly the same input, so the comparisons of Sections 4 and 6 only reflect the models. Third, the fitted pipeline is a single object that can be saved and used to serve predictions on raw data.

Moving feature engineering into the pipeline does not change any result: the scores of Table 3 are identical with both versions of the code.

6 Hyperparameter Tuning

6.1 Method

We tune the three models with Optuna (`src/tune.py`). For each model, Optuna runs a study of 30 trials. Each trial proposes a set of hyperparameters, and the corresponding pipeline is scored with the same 5-fold stratified cross-validation as in Section 4, on the training set only. The objective is the mean cross-validated PR-AUC, our main metric (Section 3.1). Optuna uses its default sampler (TPE), which proposes new values close to the best trials found so far. We fix its random seed, so the study gives the same results at every run.

All models receive the same budget of 30 trials, so that each model family has the same chance to find good settings. For every model, we tune the class weight w between 1 and $n_0/n_1 = 3.18$, as announced in Section 3.2. Besides w, we tune only a few main hyperparameters per model, to keep the search space small for a budget of 30 trials. We also avoid redundant hyperparameters: `min_samples_leaf` and `min_samples_split` both limit the growth of the trees, so we tune only the first one. Table 5 gives the search space. Each range starts from the default value of the hyperparameter and extends in the direction suggested by Section 4.

Table 5: Search space of each model. The default values are those of Section 4.

Model	Hyperparameter	Default	Range	Justification of the range
All	w	1	1 to 3.18	From no weighting to full rebalancing (n_0/n_1)
Log. reg.	`C` (log scale)	1	0.001 to 10	From 1000 times smaller to 10 times larger than the default. `C` is the inverse of the regularisation strength: above 10, the regularisation almost vanishes
Random forest	`n_estimators`	100	100 to 500	From the default to 5 times the default. More trees do not cause overfitting, they only cost time
	`max_depth`	Unlimited	5 to 30	Only shallower trees than the default, which overfits (Section 4)
	`min_samples_leaf`	1	1 to 20	Only larger leaves than the default, for the same reason
	`max_features`	$\sqrt{58} \approx 7$	4 to 28	From about half to 4 times the default (58 features after encoding)
LightGBM	`n_estimators`	100	50 to 1000	From half to 10 times the default: a lower learning rate needs more trees
	`num_leaves`	31	7 to 63	From a quarter to twice the default
	`learning_rate` (log scale)	0.1	0.01 to 0.2	From 10 times smaller to twice the default

6.2 Results

Table 6 and Figure 3 compare the default and tuned versions of each model. For every model, the best values lie strictly inside their ranges, so the ranges did not limit the search. Figure 4 shows the score of every trial.

Table 6: Default and tuned models (5-fold cross-validation on the training set, threshold 0.5). Default scores come from Table 3.

Model	Version	Prec.	Recall	F1	PR-AUC	Train PR-AUC	Best hyperparameters
Log. regression	Default	0.700	0.566	0.626	0.709	0.711	
	Tuned	0.699	0.571	0.628	0.709	0.711	$w = 1.02$, `C` $= 4.05$
Random forest	Default	0.695	0.618	0.654	0.731	0.991	
	Tuned	0.666	0.761	0.710	0.809	0.886	$w = 2.05$, 400 trees, depth 18, 3 per leaf, 25 features
LightGBM	Default	0.781	0.658	0.714	0.830	0.860	
	Tuned	0.750	0.702	**0.725**	**0.832**	0.861	$w = 1.31$, 550 trees, 9 leaves, rate 0.083

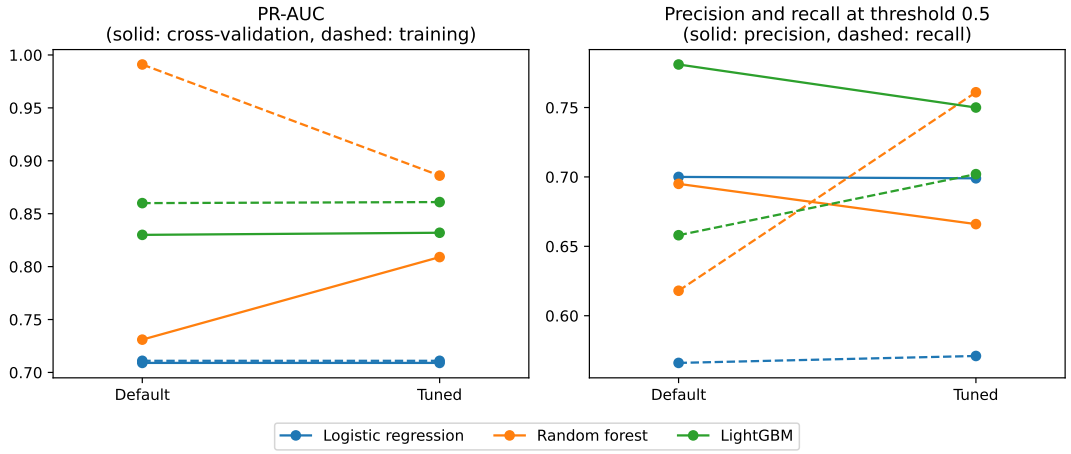


Figure 3: Default and tuned models (5-fold cross-validation). Left: cross-validated and training PR-AUC; the gap between the two shows overfitting. Right: precision and recall at the threshold of 0.5.

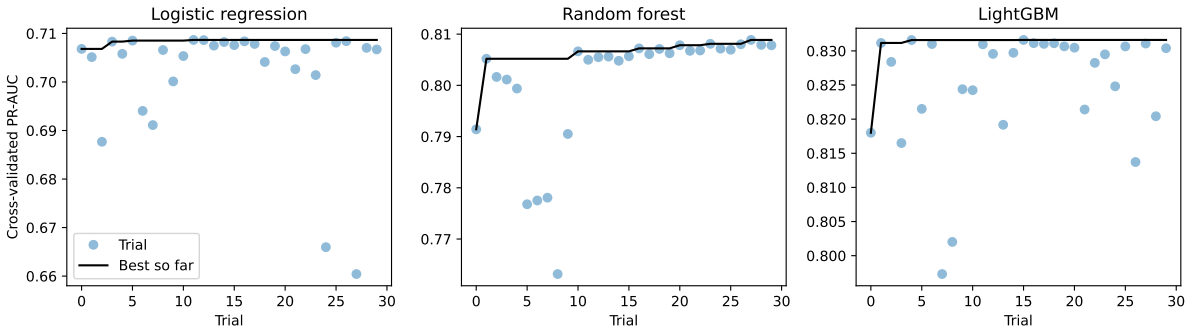


Figure 4: Cross-validated PR-AUC of each Optuna trial. The black line is the best value found so far.

Logistic regression. Tuning brings no gain: the PR-AUC stays at 0.709. Optuna also selects almost no weighting ($w = 1.02$) and a weak regularisation. This confirms the diagnosis of Section 4: the model does not overfit, and its score is limited by its linear form, which no hyperparameter can change.

Random forest. The random forest benefits most from tuning: its PR-AUC rises from 0.731 to 0.809 (Figure 3, left). The gap between training and cross-validation falls from 26 points (0.991 against 0.731) to 8 points (0.886 against 0.809). The tuned trees are shallower (depth 18) and keep at least 3 observations per leaf, so they no longer learn individual training rows, as expected in Section 4. Optuna also selects 25 features per split, far above the default of 7.

LightGBM. The PR-AUC barely changes (0.830 to 0.832): the default settings were already close to the best ones. The tuned model uses very small trees (9 leaves) with many boosting steps, and its gap between training and cross-validation stays at about 3 points (0.861 against 0.832). The main change is at the default threshold: with $w = 1.31$, recall rises from 0.658 to 0.702, while precision falls from 0.781 to 0.750, and F1 improves slightly.

Role of the class weight. Optuna selects weights between 1.02 and 2.05, below full rebalancing (3.18). Figure 3 (right) shows the effect of w: the larger the weight, the more recall rises and precision falls. The random forest has the largest weight and the largest gain in recall (0.618 to 0.761), while the logistic regression, with w close to 1, does not change. On the other hand, w has little effect on PR-AUC, which does not depend on the threshold. For example, the eight best LightGBM trials all reach a PR-AUC of 0.831 or 0.832, with w between 1.18 and 1.99. The selected value of w is therefore one of many equivalent choices, and should be read with caution.

Number of trials. Figure 4 shows that the scores level off quickly. For the logistic regression and LightGBM, the trials after the first 15 improve the best PR-AUC by less than 0.001. The random forest still improves slightly after 15 trials (+0.002), and its best trial is one of the last ones. More trials could therefore add a little to its score, but this gain is small compared with its gap to LightGBM (2.3 points). A budget of 30 trials is therefore sufficient to compare the three models.

Choice of the model. LightGBM remains the best model after tuning (PR-AUC of 0.832, against 0.809 for the random forest and 0.709 for the logistic regression), so it is our final model. This choice relies on cross-validation only. Note that the best cross-validated score is slightly optimistic: the best of 30 trials is chosen and scored on the same folds. The unbiased estimate of the final model will come from the untouched test set, which is only used in the final evaluation.

7 Experiment Tracking

All tuning runs are logged with MLflow, in a local database (`mlflow.db`). The experiment contains 94 runs, organised as follows:

- **One run per model** (3 runs). It stores the best hyperparameters and the cross-validated metrics of the best trial, and the file `best_params.json` as an artifact.
- **One run per trial** (90 runs), nested in the run of its model. Each one stores the hyperparameters of the trial, including w, and all the metrics of Section 3.1: precision, recall, F1, ROC-AUC and PR-AUC in cross-validation, the standard deviation of PR-AUC across folds, and the training PR-AUC.

- **One summary run**. It stores, as artifacts, the comparison table (`tuning_results.csv`), the best hyperparameters of the three models, and Figures 3 and 4.

This tracking makes every result of Section 6 traceable: any trial can be found with its exact settings and scores, and the trials can be compared in the MLflow interface. The best hyperparameters are also saved in `results/best_params.json`, which the next steps use to rebuild the final model.

References

[1] R. Kohavi. Scaling up the accuracy of naive-Bayes classifiers: A decision-tree hybrid. In *Proceedings of KDD*, 1996.

[2] D. B. Rubin. Inference and missing data. *Biometrika*, 63(3):581–592, 1976.

[3] T. Saito and M. Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3):e0118432, 2015.

A Additional Figures

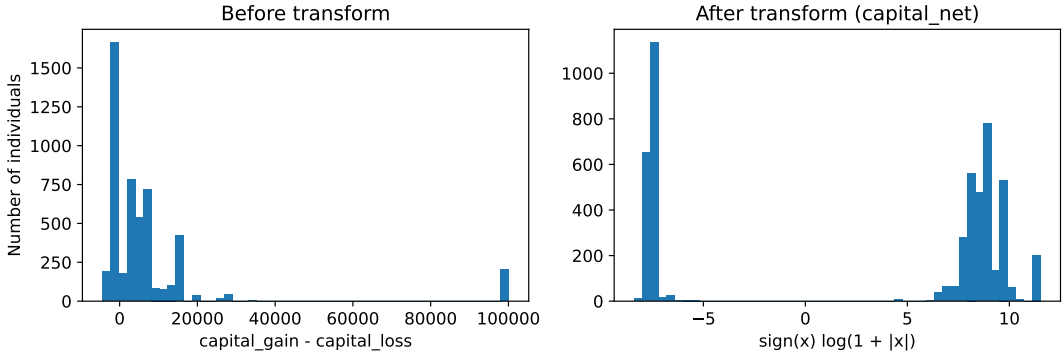


Figure 5: Non-zero net capital income (capital gain minus capital loss, training set) before and after the signed log transform. The transform compresses the long right tail; the two groups after the transform correspond to losses (negative values) and gains.